

WEEK 4

Module 1

Transitions and Animations

CSS transitions and transform functions, custom keyframe animations, and control over iteration and direction.

3 topics · [CSS Study Notes](#)

Contents

- 01 CSS Transitions and Transforms
- 02 Keyframe Animations
- 03 Animation Iteration Count and Direction

01 CSS Transitions and Transforms

Duration and timing functions · Transform functions · What can be animated

What a transition does

A transition tells the browser to move smoothly from one value to another instead of jumping. It always needs a **trigger**, such as a hover, a focus or a class added through JavaScript. Without a change of state nothing happens.

```
.btn {
  background: #1f5f9e;
  transition: background 300ms ease, transform 200ms ease;
}

.btn:hover {
  background: #16324f;
  transform: translateY(-3px);
}
```

WHERE THE TRANSITION IS DECLARED

The transition belongs on the **base state**, not inside the hover rule. Declaring it on the base state means the animation runs in both directions, on the way in and on the way out. Putting it inside `:hover` gives a smooth entry and an abrupt exit.

The four sub-properties

Property	Purpose
<code>transition-property</code>	Which property to animate, or <code>all</code> .
<code>transition-duration</code>	How long it takes, in <code>s</code> or <code>ms</code> . The default is 0s, which is why a transition with no duration appears not to work.
<code>transition-timing-function</code>	The rate of change over the duration.
<code>transition-delay</code>	How long to wait before starting.

```
transition: transform 300ms ease-in-out 100ms;
/*      property duration timing delay */
```

In the shorthand the **first** time value is always the duration and the second is the delay. Reversing them is a common error.

Timing functions

Value	Feel
<code>linear</code>	A constant rate throughout. Suits colour changes and continuous rotation, but looks mechanical for movement.
<code>ease</code>	The default. Starts quickly, then slows towards the end.
<code>ease-in</code>	Slow at the start. Best for elements leaving the screen.
<code>ease-out</code>	Fast at the start, gentle at the end. Best for elements entering, and the most natural choice for interface work.
<code>ease-in-out</code>	Slow at both ends. Suits movement between two resting positions.
<code>cubic-bezier(...)</code>	A custom curve. Values above 1 create an overshoot, which produces a springy effect.
<code>steps(n)</code>	Jumps in discrete stages rather than flowing, used for sprite sheets and typewriter effects.

Transform functions

```

transform: translateX(20px);      /* right along the x-axis      */
transform: translateY(-10px);     /* upwards along the y-axis   */
transform: translateZ(50px);      /* towards the viewer, needs perspective */
transform: translate(20px, -10px); /* both axes in one function  */
transform: translate3d(10px, 20px, 0);

transform: rotate(45deg);         /* clockwise in the plane     */
transform: rotateX(60deg);        /* tips forward or backward   */
transform: rotateY(180deg);       /* the card-flip rotation     */

transform: scale(1.1);            /* both axes                  */
transform: scaleX(1.5);
transform: skew(10deg, 4deg);

```

- Several functions may be combined in one declaration: `transform: translateY(-4px) scale(1.05) rotate(2deg)` .
- **Order matters.** The functions are applied from left to right, so rotating and then translating gives a different result from translating and then rotating.
- A second `transform` declaration **replaces** the first rather than adding to it, which is why every function must be repeated in the new rule.
- `transform-origin` moves the pivot point, which is the centre by default. `transform-origin: left bottom` makes an element swing from its corner.
- The Z axis and `rotateX` or `rotateY` only look three-dimensional when a `perspective` is set on the parent, for example `perspective: 800px` .

What can and cannot be transitioned

A property can be animated only if the browser can calculate the values in between. Colours, lengths, opacity and transforms all work. Values such as `display` , and any switch between `auto` and a fixed length, cannot be interpolated.

TWO RULES FOR SMOOTH MOTION

Animate `transform` and `opacity` wherever possible, because these can be handled by the compositor without recalculating the layout. Animating `width` , `height` , `top` or `margin` forces the browser to reflow the page on every frame, which causes visible stutter on slower devices. Also note that `display: none` cannot be transitioned, so a fade-out is built with opacity and visibility instead.

QUICK RECAP

- Declare the transition on the base state and change the value on the trigger state.
- In the shorthand the first time value is the duration and the second is the delay.
- Transform functions apply in order, and a later transform declaration replaces the earlier one.

02 Keyframe Animations

@keyframes · Applying an animation · delay and fill-mode

Why keyframes exist

A transition can only move between two states and it needs a trigger. A keyframe animation defines **as many stages as required** and can run on its own as soon as the page loads, which is what makes it suitable for loaders, attention effects and entrance animations.

Defining the animation

```
@keyframes fadeInUp {
  from {
    opacity: 0;
    transform: translateY(30px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.08); }
  100% { transform: scale(1); }
}
```

`from` and `to` are simply another way of writing `0%` and `100%`. Percentages are needed as soon as there are more than two stages, and several percentages may share one block, as in `0%, 100% { opacity: 1; }`.

Applying it to an element

```
.card {
  animation-name: fadeInUp;
  animation-duration: 600ms;
  animation-timing-function: ease-out;
  animation-delay: 200ms;
  animation-fill-mode: both;
}

/* the same thing written as a shorthand */
.card { animation: fadeInUp 600ms ease-out 200ms both; }
```

As with transitions, the **first time value is the duration and the second is the delay**. An animation with no duration never plays, since the default is zero.

animation-fill-mode

Value	Behaviour outside the animation period
<code>none</code>	The default. The element uses its normal styles before the animation starts and returns to them once it ends.
<code>forwards</code>	The element keeps the styles of the final keyframe after finishing.
<code>backwards</code>	The element takes the styles of the first keyframe during the delay, before the animation starts.
<code>both</code>	Applies backwards and forwards together, which is what most entrance animations want.

THE FLASH BEFORE THE DELAY

An element with `opacity: 0` in its first keyframe and a delay of two hundred milliseconds will be fully visible for those two hundred milliseconds, then jump to invisible and fade in. The cause is the default fill mode of `none`. Setting `backwards` or `both` applies the first keyframe immediately and removes the flash.

Equally, an element that snaps back to its starting position the moment the animation ends is showing the same default at the other end, and needs `forwards`.

A practical example

```
@keyframes spin { to { transform: rotate(360deg); } }

.loader {
  width: 32px; height: 32px;
  border: 3px solid #dfe6ef;
  border-top-color: #1f5f9e;
  border-radius: 50%;
  animation: spin 800ms linear infinite;
}
```

Only the final state has been declared, because the browser uses the element's current styles for the missing `from` block. A spinner uses `linear`, since any easing would make the rotation appear to stumble on each revolution.

QUICK RECAP

- Keyframes allow many stages and can run without any trigger.
- `from` and `to` are equivalent to 0 per cent and 100 per cent.
- `fill-mode` decides what the element looks like before and after the animation runs.

03 Animation Iteration Count and Direction

Repeating · Reversing · play-state and reduced motion

animation-iteration-count

```
animation-iteration-count: 1;           /* the default           */
animation-iteration-count: 3;          /* runs three times      */
animation-iteration-count: infinite;   /* never stops           */
animation-iteration-count: 0.5;        /* stops halfway through */
```

Fractional values are permitted and stop the animation part of the way through its cycle, which is occasionally useful for a partial reveal.

animation-direction

Value	Behaviour
<code>normal</code>	The default. Every cycle runs from 0 per cent to 100 per cent, snapping back to the start each time.
<code>reverse</code>	Every cycle runs from 100 per cent to 0 per cent.
<code>alternate</code>	The first cycle runs forwards, the second backwards, and so on.
<code>alternate-reverse</code>	Begins backwards and then alternates.

WHY ALTERNATE MATTERS

A pulse written as `0% scale(1)` and `100% scale(1.1)` with `infinite` alone will grow smoothly and then snap back to its small size, which reads as a jolt. Adding `alternate` makes it shrink back smoothly, so the middle keyframe is no longer needed and the animation halves in length.

```
/* both produce the same smooth breathing effect */
@keyframes breathe { 0%,100% { transform: scale(1); } 50% { transform: scale(1.1); } }
.a { animation: breathe 2s ease-in-out infinite; }

@keyframes grow { to { transform: scale(1.1); } }
.b { animation: grow 1s ease-in-out infinite alternate; }
```

The full shorthand

```
animation: slide 2s ease-in-out 500ms infinite alternate both;
/*      name dur timing  delay  count  direction fill */
```

The order of the keywords is flexible, but the two time values are read positionally: duration first, delay second.

animation-play-state

```
.marquee { animation: scroll 20s linear infinite; }
.marquee:hover { animation-play-state: paused; }
```

Pausing freezes the animation at its current point and resuming continues from there rather than restarting, which is why this is the correct way to give users control over continuous motion.

Respecting reduced motion

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

THIS IS NOT OPTIONAL

Continuous or large-scale motion can trigger nausea, dizziness and migraine in people with vestibular disorders. The operating system exposes the user's preference, and honouring it takes only the block above. Infinite animations are the most important ones to disable.

Points worth remembering

- An infinite animation continues to consume resources even when scrolled out of view, so it should be paused or removed when it is not visible.
- Applying two animations to one element is written as a comma-separated list, in which every sub-property is matched positionally.
- Re-triggering an animation from JavaScript requires removing the class, forcing a reflow and then adding it again, since simply re-adding the class does nothing.

QUICK RECAP

- iteration-count accepts a number or infinite, and fractions are allowed.
- alternate reverses every second cycle and removes the snap-back.
- Always provide a prefers-reduced-motion fallback for continuous animation.